Laboratory 1: Getting Started with React Native (Expo + Tailwind)

Name: _____ Section: _____ Date: _____

Estimated time: 2 to 3 hours

1. Overview

In this laboratory you will set up a React Native development environment, style your app with Tailwind CSS (through a library called NativeWind), build a Hello World app, and then redesign it into a simple mobile screen with a top navigation bar, texts, buttons, and a bottom navigation bar.

At the end of this lab the app will look like a real mobile app, but the buttons and navigation items will not do anything yet. Behavior (state, events, and screen navigation) comes in the next laboratory.

2. Learning Objectives

After completing this laboratory, you should be able to:

1. Install and verify the tools needed for React Native development using Expo.
2. Create, run, and preview a React Native project on an Android emulator.
3. Install and configure NativeWind so Tailwind utility classes work in React Native.
4. Explain the difference between HTML tags and React Native core components.
5. Build a static screen using `View`, `Text`, `Pressable`, `ScrollView`, and `SafeAreaView`.
6. Design a top bar and a bottom navigation bar using layout classes such as `flex-row` and `flex-1`.

3. Requirements

This lab runs on the Android emulator, so no QR code scan is needed. A physical phone with Expo Go (Step A4) or the web browser preview (press `w` after starting the dev server) can be used as alternatives.

Item	Details
Computer	Windows, macOS, or Linux with internet access (8 GB of RAM or more is recommended for the emulator)
Node.js	Current LTS version (v22.13 or newer)
Code editor	Visual Studio Code
Git	Optional but recommended
Android Studio	With an Android Virtual Device (emulator); you will set this up in Part A
Physical phone	Optional alternative: Android or iOS device with the Expo Go app (scan a QR code)
Wi-Fi	Only for the optional phone setup: phone and computer on the same network

Which Expo SDK? This lab was tested on **Expo SDK 57** with **NativeWind 4.2.7** and **Tailwind CSS 3.4**. Expo SDK 58 is also available, but NativeWind v4 has not been confirmed on it. Use SDK 57 unless your instructor says otherwise.

4. Background: Key Ideas Before You Start

React Native lets you build mobile apps using JavaScript and React. Instead of rendering HTML in a browser, it renders real native UI elements on Android and iOS.

Expo is a toolset built around React Native. It handles the heavy setup (Android and iOS build tools) so you can run your app instantly on an Android emulator or a phone.

Tailwind CSS / NativeWind is a set of ready-made utility classes such as `bg-blue-500`, `p-4`, and `text-xl`. NativeWind lets you use those same class names in React Native through a `className` prop.

React Native has no HTML tags

There is no `div`, `p`, `h1`, or `button` in React Native. You use core components instead:

Web (HTML)	React Native	Purpose
`<div>`	`<View>`	Container / layout box
`<p>`, `<h1>`, `<span>`	`<Text>`	All text must be inside `<Text>`
`<button>`	`<Pressable>` or `<Button>`	Something the user can press
`<img>`	`<Image>`	Pictures
scrollable `<div>`	`<ScrollView>`	Scrollable content
(none)	`<SafeAreaView>`	Keeps content away from the notch and status bar

Two more differences to remember:

- A `View` lays out its children in a column by default (like `flex-col`), so you add `flex-row` when you want items side by side.
- Styles do not cascade. A `text-white` on a `View` will not color the `Text` inside it. Put text classes on the `Text` itself.

Part A: Environment Setup

Step A1: Install Android Studio

Android Studio gives you the Android emulator, the Android SDK, and the `adb` tool that Expo uses to talk to the emulator.

1. Download Android Studio from developer.android.com/studio and install it using the Standard setup.
2. Open Android Studio once and wait until it finishes downloading the Android SDK and its components.
3. Open Settings > Languages & Frameworks > Android SDK (on macOS: Android Studio > Settings) and copy the Android SDK Location. You need this path in the next step.

Step A2: Set the Android environment variables

Expo needs to know where the Android SDK is.

Windows

1. Search for "Edit environment variables for your account" and open it.
2. Click New and add a user variable named `ANDROID_HOME` whose value is the SDK location you copied (usually `C:\Users\<your-name>\AppData\Local\Android\Sdk`).
3. Select the `Path` variable, click Edit, and add `%ANDROID_HOME%\platform-tools`.

macOS / Linux: add these lines to `~/.zshrc` (macOS) or `~/.bashrc` (Linux). Change the SDK path if yours is different (on Linux it is usually `$HOME/Android/Sdk`).

```
export ANDROID_HOME=$HOME/Library/Android/sdk
export PATH=$PATH:$ANDROID_HOME/platform-tools
```

Close and reopen your terminal, then verify:

```
adb --version
```

A version number should be printed.

Step A3: Create and start an emulator

1. In Android Studio, open Device Manager (on the welcome screen: More Actions > Virtual Device Manager).
2. Click Create Device, choose a recent phone (for example Pixel 7), and click Next.
3. Pick a system image of a recent Android version (API 34 or newer). Click Download beside it if needed, then click Next and Finish.
4. Press the Play button next to your new device and wait until the Android home screen appears. The first start can take a few minutes.

If the emulator will not start, turn on virtualization (Intel VT-x or AMD-V) in your BIOS. On Windows, also enable Windows Hypervisor Platform.

Step A4: Optional alternative: a physical phone with Expo Go (QR code)

This lab uses the emulator, so you may skip this step. If you would rather test on your own phone, install Expo Go from the Play Store (Android) or App Store (iOS) and connect the phone to the same Wi-Fi network as your computer. You will scan a QR code in Step B3.

Note: the store version of Expo Go is moving to SDK 58 only. An SDK 57 project may not open on it, so the emulator is the safer choice.

Part B: Create Your First React Native Project

Step B1: Generate the project

Open your terminal in the folder where you keep your projects and run this as a single line:

```
npx create-expo-app@latest lab1-hello-rn --template blank@sdk-57
```

If the command asks which SDK to use, choose SDK 57. When it finishes, go inside the project and open it in VS Code:

```
cd lab1-hello-rn
code .
```

Step B2: Look around the project

Identify these files in the VS Code explorer:

File / Folder	What it does
`App.js`	The main component. This is the screen you will edit.
`app.json`	Project settings (name, icon, platforms).
`package.json`	List of dependencies and scripts.
`assets/`	Images and icons.
`node_modules/`	Installed packages (never edit or submit this folder).

If your project has `App.tsx` instead of `App.js`, that is fine. Use that file wherever this lab says `App.js`.

Step B3: Start the development server

```
npx expo start
```

Metro (the bundler) starts and prints a menu of keyboard shortcuts. Make sure your emulator from Step A3 is already running on its home screen, then press the `a` key in the terminal.

Expo opens the project in the Android emulator. The first launch can take a few minutes because Expo installs its Expo Go viewer app on the emulator automatically. You do not need to scan anything.

You should see the default text "Open up App.js to start working on your app!" inside the emulator.

Other ways to preview (optional): press `w` to open the web browser preview. With a physical phone and Expo Go, scan the QR code shown in the terminal instead (Android: open Expo Go and tap Scan QR code; iOS: use the Camera app). If the phone cannot connect, stop the server and run `npx expo start --tunnel`.

Part C: Hello World

Open `App.js`, delete everything, and replace it with:

```
import { View, Text } from 'react-native';

export default function App() {
  return (
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center'
}}>
      <Text>Hello, World!</Text>
    </View>
  );
}
```

Save the file. The app in the emulator reloads automatically (this is called Fast Refresh).

Observe and answer (write in Section 8):

1. What happens if you remove the `<Text>` tag and put the words "Hello, World!" directly inside the `<View>`?
2. What does `flex: 1` do to the `View`?

Part D: Install and Configure Tailwind (NativeWind)

In Part C you styled the screen with a style object. Now you will set up Tailwind so you can use class names instead. This setup follows the official NativeWind v4 installation steps for Expo.

Stop the dev server first (press Ctrl + C in the terminal).

Step D1: Install the packages

Run these commands one at a time. Each command must be on a single line, so if your copy wraps onto two lines, join them before pressing Enter.

First, NativeWind itself:

```
npm install nativewind@4.2.7
```

Second, the development packages:

```
npm install --save-dev tailwindcss@^3.4.17 prettier-plugin-
tailwindcss@^0.5.11 babel-preset-expo
```

Third, install the packages that depend on your Expo SDK using `expo install`, not `npm install`. Plain `npm install` fetches the newest versions, which do not match SDK 57 and cause peer dependency warnings or crashes. `react-native-reanimated` also needs `react-native-worklets`:

```
npx expo install react-native-reanimated react-native-worklets react-native-
safe-area-context
```

Finally, let Expo check that every package matches your SDK, and accept any fixes it offers:

```
npx expo install --check
```

Keep `nativewind@4.2.7` and `tailwindcss@^3.4.17` exactly as written. NativeWind v4 requires Tailwind CSS v3. Do not install NativeWind v5 (a pre-release); it uses Tailwind CSS v4 and a different setup.

After installing you may see "vulnerabilities" and a `DEP0151` deprecation warning. Both are normal for React Native projects. Do not run `npm audit fix --force`, because it can upgrade packages to versions that break the setup.

Step D2: Create the Tailwind config

```
npx tailwindcss init
```

Open the new `tailwind.config.js` and replace its contents with:

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  // NOTE: Update this to include the paths to all files that contain
Nativewind classes.
  content: ['./App.{js,jsx,ts,tsx}', './components/**/*.{js,jsx,ts,tsx}'],
  presets: [require('nativewind/preset')],
  theme: {
    extend: {},
  },
  plugins: [],
};
```

The `content` array tells Tailwind which files to scan for class names. If a file is not listed, its classes will not work. The pattern `App.{js,jsx,ts,tsx}` matches your App file whether it is JavaScript or TypeScript.

Step D3: Create the global CSS file

Create a new file named `global.css` in the project root (same level as `App.js`) and add:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Step D4: Create the Babel config

Create `babel.config.js` in the project root:

```
module.exports = function (api) {
  api.cache(true);
  return {
    presets: [
      ['babel-preset-expo', { jsxImportSource: 'nativewind' }],
      'nativewind/babel',
    ],
  };
};
```

Step D5: Create the Metro config

Create `metro.config.js` in the project root:

```
const { getDefaultConfig } = require('expo/metro-config');
const { withNativeWind } = require('nativewind/metro');

const config = getDefaultConfig(__dirname);

module.exports = withNativeWind(config, { input: './global.css' });
```

Step D6: Allow the web preview to use Metro

Open `app.json` and make sure the `web` section (inside `expo`) contains the Metro bundler setting:

```
{
  "expo": {
    "web": {
      "bundler": "metro"
    }
  }
}
```

Keep any other settings that are already inside `expo` and `web`. Only add the missing parts.

Step D7: Import the CSS in your app and test that Tailwind works

The CSS file must be imported once, at the top of your App file. Replace `App.js` with:

```
import './global.css';
import { View, Text } from 'react-native';

export default function App() {
  return (
    <View className='flex-1 items-center justify-center bg-indigo-600'>
      <Text className='text-4xl font-bold text-white'>Hello, Tailwind!</Text>
    </View>
  );
}
```

Start the server with a cleared cache (important after changing config files):

```
npx expo start -c
```

You should see white bold text on an indigo background. If the screen is plain white with small black text, Tailwind is not active yet. Check the Troubleshooting section.

Part E: Build a Static App Screen

Now you will turn the Hello World into a mobile-style screen. Nothing needs to work when pressed. You are only building the layout.

Target layout

```
+------------------------------+
| My First App               ☰ |  <- Top navigation bar
+------------------------------+
|                              |
|        Hello, World!         |  <- Heading text
|   Welcome to React Native... |  <- Paragraph text
|                              |
|       [ Get Started ]        |  <- Primary button
|       [ Learn More ]         |  <- Secondary button
|                              |
+------------------------------+
| Home      Search      Me     |  <- Bottom navigation bar
+------------------------------+
```

Step E1: Plan the file structure

Do not put everything in `App.js`. Each part of the screen gets its own component file. In the project root (next to `App.js`), create a folder named `components`, then create these two files inside it:

File	What it contains
`components/TopBar.js`	The top navigation bar: the app title on the left and a menu icon on the right
`components/NavBar.js`	The bottom navigation bar with three items: Home, Search, and Profile

`App.js` stays as the main screen that assembles everything. Because `tailwind.config.js` already scans `./components/**/*`, Tailwind classes inside these files will work.

Step E2: Import before you use

In a React web project you can write `<div>`, `<p>`, or `<button>` without importing anything, because the browser already knows those tags. React Native has no built-in tags like that. Every tag you use must be imported at the top of every file where you use it. If you forget, the app shows a red error such as `Property 'View' doesn't exist` or `Can't find variable: Text`.

Follow these import rules:

- Components from a library go inside curly braces, separated by commas: `import { Name1, Name2 } from 'package-name'`.

Component	Import it from	Used for
`View`, `Text`, `Pressable`, `ScrollView`	`'react-native'`	Boxes, text, pressable areas, scrolling
`SafeAreaProvider`, `SafeAreaView`	`'react-native-safe-area-context'`	Keeping content clear of the status bar and camera notch
`TopBar`, `NavBar` (your own files)	`'./components/TopBar'` and `'./components/NavBar'`	Your own reusable components

- Your own components use a default export in their file (`export default function Name()`) and are imported without curly braces: `import Name from './path'`.
- Import only the tags that the file actually uses. `TopBar.js` does not need `Pressable`; `NavBar.js` does.
- `import './global.css'` is written once, in `App.js` only.

Step E3: Build the TopBar component

Open `components/TopBar.js` and do the following:

1. Import the tags this file uses (see Step E2).

2. Create a function named `TopBar` that returns the bar, and export it as the default export.
3. The outer tag is a `View` that acts as the bar. Style it so that it has a solid background color, some horizontal and vertical padding, and places its children side by side with space between them. Classes to look up in the cheat sheet (Section 7): `flex-row`, `items-center`, `justify-between`, `bg-...`, `px-...`, `py-...`.
4. Inside the bar, add two `Text` tags: the app title (large, bold, white) and a menu symbol such as ☰ on the right.
5. Remember: text styles such as `text-white` and `text-xl` go on the `Text` tag, not on the `View`.

Step E4: Build the NavBar component

Open `components/NavBar.js` and do the following:

1. Import `View`, `Text`, and `Pressable` from `react-native`.
2. Create a function named `NavBar` and export it as the default export, the same way as `TopBar`.
3. The outer `View` lays out its children in a row, has a white background, and has a thin top border that separates it from the content (`flex-row`, `border-t`, `border-slate-200`).
4. Create three items using `Pressable`: Home, Search, and Profile. Give each one `flex-1` so the three share the width equally, and center its content (`items-center`, `py-2`).
5. Each item contains two `Text` tags stacked vertically: an emoji icon on top (for example 🏠, 🔍, 👤) and a small label below it.
6. Make Home look selected by giving its label a different color and a bolder weight than the other two.
7. Optional: to avoid repeating the same markup three times, write a small helper function in the same file that receives the icon and label as props, the same way props work in ReactJS.

Step E5: Assemble the screen in App.js

Open `App.js`, remove the Tailwind test code from Step D7, and build the screen:

1. Keep `import './global.css'` at the top. Then import `Text`, `Pressable`, and `ScrollView` from `react-native`, import `SafeAreaProvider` and `SafeAreaView` from `react-native-safe-area-context`, and import your two components `TopBar` and

`NavBar`.

2. Make `App` return a `SafeAreaProvider` that wraps a `SafeAreaView`. Give the `SafeAreaView` the class `flex-1` and a background color so it fills the whole screen.
3. Inside the `SafeAreaView`, place three things from top to bottom: `<TopBar />`, then a `ScrollView`, then `<NavBar />`. Give the `ScrollView` the class `flex-1` so it takes all the space between the two bars; this keeps the navigation bar pinned at the bottom. Use `contentContainerClassName` with a padding class for the space around its content.
4. Inside the `ScrollView`, add a heading `Text` ("Hello, World!": large, bold, dark color) and a paragraph `Text` below it (smaller, grayer, with a little top margin).
5. Below the paragraph, add two buttons. Each button is a `Pressable` that contains a `Text`; the label is never written directly inside the `Pressable`. Style the first as a filled primary button and the second as an outlined secondary button. For both, use rounded corners, vertical padding, `items-center`, and a top margin to separate them.
6. Optional: add `active:opacity-80` to a `Pressable` to make it fade slightly while it is pressed.

Why `Pressable` and not `Button`? React Native's built-in `Button` cannot be styled with `className`, while `Pressable` is a blank container you can design freely.

The buttons and navigation items do nothing yet. That is expected. Do not add `onPress` in this laboratory.

Part F: Your Turn (Customization Tasks)

Modify the app so it is clearly yours. Complete all of the following:

1. Change the app title in the top bar (`TopBar.js`) to your own app name.
2. Change the top bar and primary button color from indigo to another Tailwind color (for example `emerald`, `rose`, or `sky`).
3. Replace the paragraph with a short description about yourself (name, course, and one hobby).
4. Add a third button below "Learn More" with a different style (for example a red "Delete" look using `bg-red-500`).
5. Add a fourth item to the bottom navigation bar (`NavBar.js`), for example "Settings" with a ⚙️ icon.
6. Make the heading and paragraph centered using `text-center`.

7. Make a different item look selected in the bottom navigation bar.

5. Troubleshooting

Problem	What to try
`npx` or `node` is not recognized	Close and reopen the terminal; reinstall Node.js and tick the option to add it to PATH.
`react-native-worklets: command not found` (or similar "command not found" for a package name)	An install command wrapped onto two lines and the second line ran as its own command. Run each command as one single line.
The emulator is very slow or will not start	Turn on virtualization (Intel VT-x or AMD-V) in the BIOS; on Windows enable Windows Hypervisor Platform; close heavy programs; pick a lighter device in Device Manager.
Pressing `a` says no Android device was found, or `adb` is not recognized	Start the emulator first and wait for its home screen; check `ANDROID_HOME` and the platform-tools path (Step A2); reopen the terminal; run `adb devices` and make sure the emulator is listed.
The app is stuck on "Opening project" in the emulator	Press `a` again, or restart with `npx expo start -c`. If it still fails, cold boot the emulator from Device Manager.
(Optional phone) The phone cannot open the app	Make sure both devices use the same Wi-Fi, disable VPNs, or run `npx expo start --tunnel`.
Tailwind classes do nothing (plain white screen)	Confirm `import './global.css';` is in `App.js`, the file paths in the `tailwind.config.js` content array match your files, then run `npx expo start -c`.
`Cannot find module 'nativewind/babel'`	NativeWind did not install. Re-run the install commands in Step D1 from inside the project folder.

Problem	What to try
Red error screen mentioning `react-native-reanimated` or `worklets`	Run `npx expo install react-native-worklets`, then `npx expo install --fix`, then `npx expo start -c`.
Warnings about vulnerabilities or `DEP0151`	These are normal. Ignore them, and do not run `npm audit fix --force`.
"Text strings must be rendered within a `<Text>` component"	All words must be inside a `<Text>` tag.
`Property 'View' doesn't exist`, `Can't find variable: Text`, or "Element type is invalid"	A tag is missing its import, or your component file is missing `export default`. Check the import lines (Step E2).
Changes do not appear	Save the file, then press `r` in the terminal to reload the app.
The website preview shows a blank page	Check that `app.json` has the web bundler set to `metro` and restart with `-c`.
PowerShell blocks scripts (Windows)	Use Command Prompt instead, or ask your instructor how to allow scripts.

6. Grading Rubric (100 points)

Criteria	Points
Environment verified and project created	15
NativeWind and Tailwind correctly configured	20
Static screen has top bar, text, buttons, and bottom bar	30
All seven customization tasks completed	20
Separate TopBar and NavBar files, correct imports, readable code	5
Answers to questions in Section 8	10

7. Common Class Cheat Sheet

Goal	Classes
Fill available space	`flex-1`
Row layout	`flex-row`
Center content	`items-center justify-center`
Space between items	`justify-between`, `gap-2`
Padding / margin	`p-4`, `px-4`, `py-2`, `m-2`, `mt-4`
Background color	`bg-white`, `bg-slate-50`, `bg-indigo-600`
Text size	`text-sm`, `text-base`, `text-xl`, `text-3xl`
Text weight / color	`font-bold`, `font-semibold`, `text-white`, `text-slate-600`
Rounded corners	`rounded`, `rounded-xl`, `rounded-full`
Border	`border`, `border-t`, `border-slate-200`
Pressed style	`active:opacity-80`

8. Questions

Answer briefly in your own words.

1. What happens when you place plain text directly inside a `<View>` without a `<Text>`? Why?
2. What does `flex: 1` (or `flex-1`) do?
3. Name the React Native equivalents of the HTML tags `div`, `p`, `img`, and `button`.
4. Why do we need a `global.css` file and a `tailwind.config.js` file in a NativeWind project?
5. Why is `Pressable` preferred over `Button` when you want custom styling?
6. In your layout, what is the difference between `flex-row` and the default direction of a `View`?
7. Your buttons currently do nothing. What do you think needs to be added to make them respond when pressed? (You will implement this in the next laboratory.)

8. In a web React project you can use `<div>` without importing it. Why must `View` and `Text` be imported in React Native, and which package provides them?
9. Why were `TopBar` and `NavBar` placed in their own files, and which two statements make it possible to use them inside `App.js`?

Submission: Submit your answers in Google Classroom. Name your file using this format: `Surname_ReactNativeQuestionnare` (for example, `Santos_ReactNativeQuestionnare`).